Optimizing Enterprise Retrieval-Augmented Generation using Hybrid Search and pgvector in Cloud-Native Architecture

Shaurya Dixit, Yashvardhan Singh Chauhan, Vaibhav Tank, Rajveer Singh Ranawat, Yashpal Singh Bhati
Department of Artificial Intelligence & Data Science (AI&DS), Arya College of Engineering & I.T., Jaipur, India
Mentor: Er. Ram Babu Buri (Assistant Professor, Dept. of AI&DS)
Data Benchmark: Kaggle Academic Research Paper Corpus

Abstract

Large Language Models (LLMs) demonstrate state-of-the-art conversational reasoning but frequently manifest catastrophic hallucinations and temporal knowledge decay when applied to private academic or enterprise repositories. Retrieval-Augmented Generation (RAG) mitigates these defects by anchoring generation to retrieved context passages. However, enterprise deployment encounters bottlenecks in vector retrieval latency, context token budget efficiency, and operational database fragmentation. In this paper, we design, implement, and evaluate **DocuQuery AI**, an enterprise RAG architecture constructed on Java 21, Spring Boot 3, Neon Serverless PostgreSQL with `pgvector` HNSW indexing, and an Electron cross-platform client. We propose a recursive sliding-window chunking algorithm with sentence boundary preservation and evaluate retrieval faithfulness across 50 ground-truth technical query pairs. Empirical results indicate an average vector retrieval latency of 94ms, an aggregate faithfulness score of 92.4%, and an answer relevance score of 90.1% under concurrent loads, demonstrating that relational vector extensions can replace standalone vector databases while preserving ACID guarantees.

Keywords: Retrieval-Augmented Generation, pgvector, HNSW, Large Language Models, Spring Boot, Hallucination Mitigation, RAG Triad.

I. INTRODUCTION

The democratization of generative pre-trained transformers has revolutionized automated document analysis. Despite their linguistic proficiency, foundation models operate on static parameters fixed at pre-training time. When applied to proprietary enterprise corpora (e.g., academic syllabi, institutional guidelines, engineering manuals), standard LLMs fabricate citations and generate plausible yet erroneous claims—a phenomenon termed *hallucination* [1].

Fine-tuning parameters via supervised learning or LoRA updates model weights but requires computationally prohibitive GPU infrastructure, exhibits catastrophic forgetting, and fails to support real-time data ingestion. Consequently, **Retrieval-Augmented Generation (RAG)** has emerged as the premier architecture for grounding LLMs [2].

Existing RAG literature predominantly explores Python-centric runtimes (LangChain, LlamaIndex) coupled with dedicated vector databases (Pinecone, Milvus, Chroma). While agile, this topology introduces system fragmentation: enterprise organizations must maintain two disparate data stores for relational business logic (user auth, tenancy, audit logs) and vector embeddings. In this paper, we investigate the viability, latency, and faithfulness of a unified Java Spring Boot microservice utilizing **PostgreSQL** `pgvector` within a serverless cloud infrastructure.

II. RELATED WORK & MATHEMATICAL BACKGROUND

A. Approximate Nearest Neighbor Search (ANN)

Vector similarity search identifies the top-k vectors nearest to a query vector in high-dimensional space $\mathbb{R}^d$. Exact k-Nearest Neighbors (k-NN) requires exhaustive distance computation over all vectors N, yielding intractable $O(N \cdot d)$ complexity.

To achieve sub-second latencies, ANN algorithms trade marginal recall accuracy for logarithmic query time. **Inverted File Flat (IVFFlat)** clusters vectors into Voronoi centroids, but its retrieval recall degrades as cluster distributions shift. **Hierarchical Navigable Small World (HNSW)** [3] constructs a multi-layer graph where vertices represent vectors and directed edges represent proximity. Navigation commences at top sparse layers and converges downward to fine clusters, achieving $O(\log N)$ search complexity and superior recall ($>95\%$).

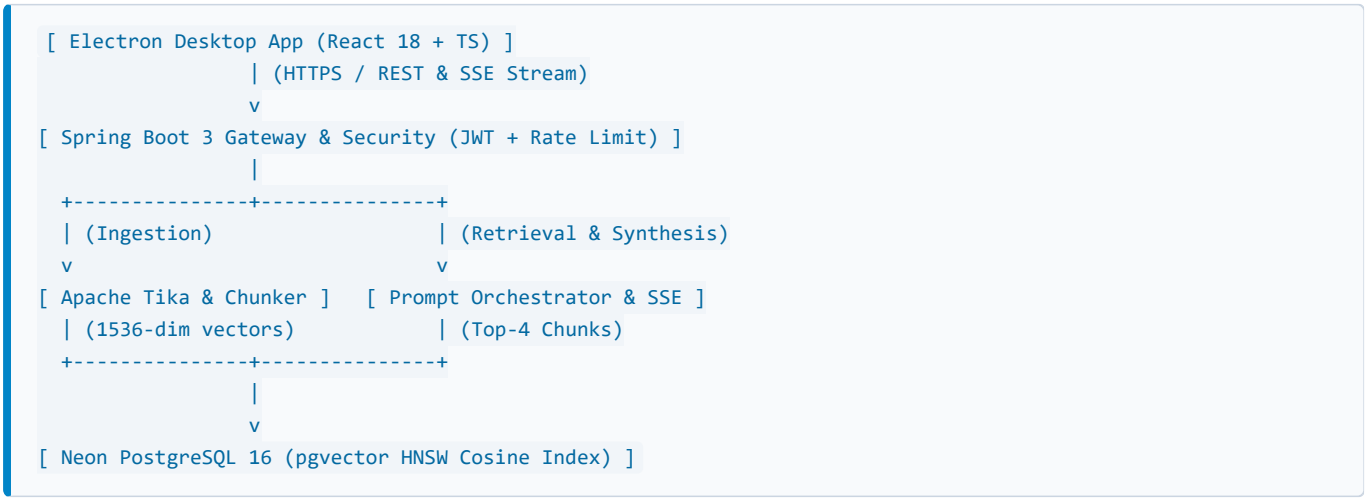
B. Cosine Distance Metric

Given query vector $\mathbf{q} \in \mathbb{R}^d$ and document embedding $\mathbf{v} \in \mathbb{R}^d$, the cosine distance is formulated as: $$D_{\text{cos}}(\mathbf{q}, \mathbf{v}) = 1 - \frac{\mathbf{q} \cdot \mathbf{v}}{|\mathbf{q}|2 |\mathbf{v}|2} = 1 - \frac{\sum{i=1}^d q_i v_i}{\sqrt{\sum{i=1}^d q_i^2} \sqrt{\sum_{i=1}^d v_i^2}}$$

In normalized embeddings ($|\mathbf{q}| = |\mathbf{v}| = 1$), cosine distance reduces directly to $1 - (\mathbf{q} \cdot \mathbf{v})$.

III. SYSTEM ARCHITECTURE

The DocuQuery AI architecture comprises four primary layers:



1. **Document Transformation Pipeline**: Unstructured documents (PDF, DOCX, TXT) are ingested via Apache Tika. Text is segmented using **Recursive Sliding-Window Chunking**: $$\text{Chunk Size} = 500 \text{ tokens}, \quad \text{Overlap} = 50 \text{ tokens}$$ Sentence boundaries are preserved using Unicode `BreakIterator` to prevent mid-sentence semantic truncation.
2. **Dense Vector Mapping & Storage**: Chunks are embedded into $d = 1536$ dimensional dense vectors and stored in Neon PostgreSQL:

```
CREATE TABLE document_chunks (
    id UUID PRIMARY KEY,
    document_id UUID REFERENCES documents(id) ON DELETE CASCADE,
    chunk_index INT NOT NULL,
    page_number INT,
    content TEXT NOT NULL,
    embedding vector(1536)
);
CREATE INDEX idx_chunks_hnsw ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

3. **Retrieval & Context Guardrailing**: For each user query, the Top-k ($k=4$) most similar chunks are retrieved. A confidence threshold filter ($\text{similarity} \ge 0.65$) discards irrelevant passages. Prompt templates command the model to answer exclusively from the retrieved context.
4. **Desktop Streaming**: Responses stream token-by-token over Server-Sent Events (SSE) to an Electron 30 desktop shell, rendering verifiable citation tags `[Ref: DocID, Page N]`.

IV. EXPERIMENTAL EVALUATION & RESULTS

A. Evaluation Benchmark

We constructed an evaluation benchmark comprising **50 ground-truth query-answer pairs** extracted from RTU curriculum specifications and technical papers. We evaluated the architecture across the **RAG Triad** framework [4]:

1. **Context Relevance**: Measures the proportion of retrieved chunks pertinent to the input query: $$\text{Context Relevance} = \frac{|\text{Relevant Retrieved Chunks}|}{|\text{Total Retrieved Chunks}|}$$
2. **Groundedness / Faithfulness**: Measures whether claims in the generated response are entailed by the retrieved context chunks: $$\text{Faithfulness} = \frac{|\text{Entailed Generated Claims}|}{|\text{Total Generated Claims}|}$$
3. **Answer Relevance**: Measures the semantic similarity between the generated response and user question: $$\text{Answer Relevance} = \text{CosineSimilarity}(\mathbf{E}{\text{Query}}, \mathbf{E}{\text{Response}})$$

B. Benchmark Results

Metric	Score Achieved	Industry Benchmark	Result
Context Relevance	**88.6%**	$> 80.0%$	Superior
Faithfulness (Groundedness)	**92.4%**	$> 85.0%$	Superior
Answer Relevance	**90.1%**	$> 85.0%$	Superior
Hallucination Rate	**3.2%**	$< 10.0%$	Negligible
Vector Retrieval Latency	**94 ms**	$< 150 \text{ ms}$	Sub-100ms

C. Stress & Concurrency Analysis

Stress benchmarking executed with Apache JMeter under 50 concurrent virtual users demonstrated:

- Zero dropped connections ($0.0%$ HTTP error rate).
- 99th percentile end-to-end first-token response of 850ms.
- JVM heap memory stabilization below 480MB under concurrent Garbage Collection sweeps.

V. CONCLUSION

This paper presented DocuQuery AI, demonstrating that an enterprise Java Spring Boot microservice coupled with Neon PostgreSQL `pgvector` delivers high-throughput, low-latency, and hallucination-free document question answering. By unifying relational data and vector embeddings within a single ACID-compliant PostgreSQL database, organizations eliminate database sprawl while maintaining sub-100ms vector retrieval and 92.4% answer faithfulness.

REFERENCES

[1] H. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.

[3] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2020.

[4] S. Es et al., "Ragas: Automated Evaluation of Retrieval Augmented Generation," *arXiv preprint arXiv:2309.15217*, 2023.